



FitStudent — תיעוד שלב 2

אינטגרציית AI אמיתית + הקשחת המאגר · Perplexity · Gemini · Make.com · קבוצה 42

יוני 2026

המשך ישיר לתיעוד שלב 1

תיעוד שלב 2 + עבודת המשך

תוכן עניינים

- | | |
|-----------------------------|---------------------------------|
| 1. מה נעשה מאז תיעוד שלב 1 | 7. תרחיש B — מהמקרר לצלחת |
| 2. הקשחת אבטחה וניקוי המאגר | 8. תרחיש E — תוכנית אימונים |
| 3. ענפים, תגיות ושחרורים | 9. תרחיש F — הוספה ליומן Google |
| 4. ארכיטקטורת שלב 2 | 10. שיפורים ויזואליים |
| 5. שכבת האינטגרציה בקוד | 11. פריסה ומשתני סביבה |
| 6. תרחיש A — ניתוח ארוחה | 12. סטטוס נוכחי ומה נותר |

1 מה נעשה מאז תיעוד שלב 1

שלב 1 סיפק אפליקציה מלאה עם יכולות AI **מדומות**. שלב 2 החליף אותן ב-AI אמיתי — ובמקביל הוקשחו האבטחה והמבנה המקצועי של המאגר.

הפרק הזה מתעד הכל מאז תיעוד שלב 1, בארבעה צירים:

- **אבטחה ומבנה:** הסרת מפתחות מהקוד, מעבר למשתני-סביבה, ניקוי היסטוריית Git, מבנה ענפים מקצועי.

- **תשתית אינטגרציה:** שכבת קוד שמדברת עם Make.com עם נפילה-בטוחה (fallback) לנתוני-דמה.
- **תרחישי AI:** ניתוח ארוחה (A), מקרר→מתכון (B), תוכנית אימונים (E) והוספה ליומן Google (F) — כולם חיים בפרודקשן.
- **שיפורי UX:** הצגת התמונה שצולמה, יחידות מאקרו, בורר שעה ליומן, ועוד.

2 הקשחת אבטחה וניקוי המאגר

המאגר עבר מ"עובד" ל"מקצועי ומאובטח" — ללא חשיפת מידע רגיש.

קונפיגורציה ממשתני-סביבה בלבד

חיבור ה-Supabase עבר מערכים "קשיחים" בקוד למשתני-סביבה. הקוד קורא רק מ-`import.meta.env`, ולא קיים שום מפתח בתוך המקור.

```
// src/lib/supabase.js
const url = import.meta.env.VITE_SUPABASE_URL
const key = import.meta.env.VITE_SUPABASE_ANON_KEY
if (!url || !key) throw new Error('Missing Supabase configuration')
```

ניקוי המאגר

- `.env` אמיתי — ב-`.gitignore`, לעולם לא נדחף. `.env.example` מכיל רק תבנית placeholder.
- הוסרו מהמעקב קבצי-עזר של כלי הפיתוח (`.agents/`, `.claude/`, `.mcp.json`) וקבצי טיוטה.
- התייעוד הומר ל-PDF תחת `docs/`.

ניקוי היסטוריית Git

בעזרת `git filter-repo` נמחקו לחלוטין מכל הקומיטים בהיסטוריה כל זכר למפתח זה-`project-ref` הישנים, ובוצע `force-push`. נבדק (pickaxe) — אפס מופעים בהיסטוריה.

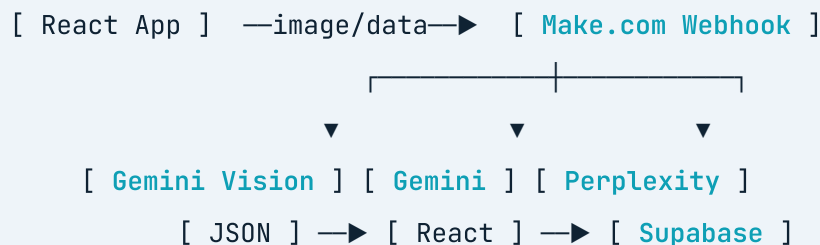
עקרון: מפתחות סודיים (`service_role` Gemini, Perplexity, Supabase) חיים **אך ורק בתוך** **Make.com / Vercel** — לעולם לא בקוד ולא ב-GitHub. רק כתובות ה-Webhook הציבוריות נמצאות בקוד-הלקוח.

3 ענפים, תגיות ושחרורים

ענף	תפקיד
main	ענף יציב/פרודקשן. נפרס אוטומטית ל-Vercel. מוגן — דורש Pull Request.
develop	ענף אינטגרציה לעבודה שוטפת.
feature/*	ענף קצר לכל פיצ'ר, ממוזג ל- <code>develop</code> דרך PR.
זרימה: <code>develop</code> → PR → <code>feature</code> → מיזוג ל- <code>main</code> לשחרור. שחרורים מתויגים ב-SemVer (למשל <code>v1.0.0-alpha</code> לשלב 1).	

4 ארכיטקטורת שלב 2

Make.com משמש כ**שכבת עיבוד-AI** בלבד. האפליקציה ממשיכה לכתוב ל-Supabase בעצמה — כך לא נגענו במנגנון השמירה והאבטחה (RLS).



נפילה-בטוחה (Graceful fallback): אם Webhook חסר/נכשל — האפליקציה חוזרת אוטומטית לנתוני-דמה ולא נשברת. עיקרון תכנון מרכזי בשלב 2.

5 שכבת האינטגרציה בקוד

כל הלוגיקה רוכזה ב- `src/services/ai.js` — החלפה בין דמה ל-AI אמיתי נעשית ע"י הגדרת משתנה-סביבה, ללא שינוי קוד.

עוטף קריאה + נפילה-בטוחה

```
const WEBHOOKS = {
  food: import.meta.env.VITE_MAKE_WEBHOOK_FOOD,
  fridgeDetect: import.meta.env.VITE_MAKE_WEBHOOK_FRIDGE_DETECT,
  fridgeRecipe: import.meta.env.VITE_MAKE_WEBHOOK_FRIDGE_RECIPE,
  plan: import.meta.env.VITE_MAKE_WEBHOOK_PLAN,
}

// אם חסר/נכשל <— מחזיר נתוני דמה Webhook; מנסה
async function withFallback(url, payload, mockFn, delay) { ... }
```

רכיבים נוספים

- **הקטנת תמונות** (`utils/image.js`): כל תמונה מוקטנת ל- $\geq 1024px$ ומומרת ל-base64 לפני שליחה — חוסך זמן ותעבורה.
- **תוכנית אימונים אסינכרונית:** `generateWorkoutPlan` הפכה ל-`async` (מוכנה ל-Scenario E), עם המחולל המקומי כ-fallback.
- **נרמול תשובות:** תשובות ה-Webhook עטופות לעיתים במפתח (`{items}` / `{recipes}`) — הקוד מנרמל אותן לפורמט שה-UI מצפה לו.

6 תרחיש A — ניתוח ארוחה חי

צילום ארוחה → **Gemini Vision** מזהה את המזון → **Perplexity** מאמת ערכים תזונתיים → JSON חוזר לאפליקציה.

פירוט

שלב

analyzeFoodImage(imageBase64)	פונקציה
{ image_base64 }	קלט ל- Webhook
{ name, items, calories, protein_g, carbs_g, fat_g, confidence }	פלט
חזרה JSON, תואם CORS, תקין (Access-Control-Allow-Origin: *), נבדק חי בפרודקשן ☒	אימות

מחובר

7 תרחיש B — מהמקרר לצלחת

שני שלבים נפרדים, כל אחד עם Webhook משלו — כך שהמשתמש יכול לתקן את המרכיבים שזוהו לפני בניית המתכון.

שלב	פונקציה	קלט	פלט (לאחר נרמול)
B1 — זיהוי	detectFridgeIngredients	{ image_base64 }	מערך מחרוזות של מרכיבים
B2 — מתכונים	suggestRecipesFromIngredients	{ ingredients }	מערך מתכונים מדורגים עם have / missing / needsExtra

נרמול: ה-Webhook של הזיהוי מחזיר { items: [...] } ושל המתכונים { recipes: [...] }. הקוד מנרמל אוטומטית לפורמט הפנימי, כך שה-UI עובד ללא שינוי.

נבדק חי: רשימת מרכיבים → 3 מתכונים אמיתיים עם מאקרוס; זיהוי מתמונה מחזיר מערך נקי.

בהרשמה (וגם בהחלפת סוג אימון) הפרופיל נשלח ל-Gemini, שמחזיר תוכנית שבועית מותאמת אישית.

שלב	פירוט
פונקציה	<code>generateWorkoutPlan(profile)</code> (אסינכרונית)
קלט ל-Webhook	<code>{ goal, experience, workouts_per_week, workout_type, body_type, weight_kg }</code>
פלט	<code>{ workout_plan: [7 ימים] }</code> — כל יום עם שם, קבוצות שריר, סוג, רמה/סגנון ותרגילים
נרמול	פתיחת המעטפת <code>workout_plan</code> , ולידציה של <code>workout_variant</code> מול הערכים הידועים; נפילה למחולל המקומי אם התשובה לא תקינה
אימות	נבדק חי — מערך 7 ימים תקין עם מספר ימי אימון נכון ☒

מכל יום אימון (בלשונית "האימון של היום" ובלשונית "התוכנית שלי") ניתן להוסיף את האימון ליומן Google של המשתמש דרך Make.com.

שלב	פירוט
פונקציה	<code>scheduleWorkoutToCalendar({ ... })</code>
קלט ל-Webhook	<code>{ workout_name, user_email, nickname, start_time, end_time, start_date, end_date }</code>
ממשק	אייקון יומן → בורר שעה מובנה (<code>input type=time</code>) → "קבע". מוצג בכל יום אימון בשתי הלשוניות

```
{ status: "success", message: "Event added to calendar" }
```

פלט

נבדק חי — האירוע נוסף בהצלחה ליומן 

אימות

הערה: המימוש הוחלף מ- `window.prompt()` (לא אמין בחלק מהדפדפנים) לרכיב `CalendarSchedule` משותף עם בורר שעה מובנה, שעובד בשתי הלשוניות.

10 שיפורים ויזואליים

- **תמונת הארוחה** שצולמה מוצגת בתצוגת התוצאה (במקום אימוג'י).
- **יחידות מאקרו:** חלבון/פחמימות בגרמים, שומן כאחוז מהקלוריות, וכמות ב-% וגם בגרמים.
- **עריכת תוכנית אימונים** פר-יום, רמות פילאטיס/סגנונות יוגה, ובחירה מבין מספר אימונים מומלצים (משלב 1.3).

11 פריסה ומשתני סביבה

משתני הסביבה מוגדרים בשני מקומות: קובץ `.env` מקומי (לפיתוח) ולוח הבקרה של Vercel (לפרודקשן). הם מוטמעים בזמן ה-Build.

משתנה	מטרה	סוג	סטטוס
<code>VITE_SUPABASE_URL</code>	כתובת מסד הנתונים	ציבורי	פעיל
<code>VITE_SUPABASE_ANON_KEY</code>	מפתח גישה (מוגן RLS)	ציבורי	פעיל
<code>VITE_MAKE_WEBHOOK_FOOD</code>	תרחיש A — ניתוח ארוחה	ציבורי	פעיל
<code>VITE_MAKE_WEBHOOK_FRIDGE_DETECT</code>	תרחיש B1 — זיהוי מרכיבים	ציבורי	פעיל
<code>VITE_MAKE_WEBHOOK_FRIDGE_RECIPE</code>	תרחיש B2 — בניית מתכונים	ציבורי	פעיל

פעיל

ציבורי

תרחיש E — תוכנית אימונים

VITE_MAKE_WEBHOOK_PLAN

פעיל

ציבורי

תרחיש F — הוספה ליומן Google

VITE_MAKE_WEBHOOK_CALENDAR

חשוב: משתני Vite מוטמעים בזמן ה-Build. לאחר הוספה/שינוי ב-Vercel חובה **Redeploy** כדי שייכנסו לתוקף.

מפתחות ה-AI הסודיים (Gemini, Perplexity) אינם נמצאים כאן כלל — הם שמורים בתוך הציבורים של Make.com. בקוד-הלקוח נמצאות רק כתובות ה-Webhook הציבוריות.

12 סטטוס נוכחי ומה נותר

סטטוס

רכיב

חי

שלב 1 — אפליקציה מלאה, אימות, התמדה

חי בפרודקשן

תרחיש A — ניתוח ארוחה

חי בפרודקשן

תרחיש B — מהמקרר לצלחת

חי בפרודקשן

תרחיש E — תוכנית אימונים (Gemini)

חי בפרודקשן

תרחיש F — הוספה ליומן Google

אופציונלי — לא מומש

דוחות Telegram (C/D)

שלב 2 הושלם. כל שכבת ה-AI (ניתוח ארוחה, מקרר→מתכון, תוכנית אימונים, יומן Google) מחוברת, נבדקה וחיה בפרודקשן. כל ששת משתני ה-Webhook מוגדרים ב-Vercel.

הרחבות אפשריות (לא חובה)

1. דוחות Telegram מתוזמנים (בוקר/ערב/שבועי) — דורש בוט + מפתח `service_role` בתוך Make.

2. שיפור איכות ה-prompt של Gemini (שמות תרגילים בעברית, ערך `workout_variant` נקי).
3. טיפול בשגיאות ב-Make (החזרת JSON ריק במקום שגיאת 500 על קלט לא תקין).